

LLM 后训练之 Model Release & Deployment Governance

模型发布、Release Gate、Canary、Rollback 与线上反馈治理

深度研究与工程手册 · 2026-08

核心命题

模型发布不是“把 checkpoint 切到 100% 流量”。它是一组可逆的风险暴露决策：候选系统必须经过离线门槛、部署仿真、有限预览、灰度/Canary、分阶段扩量、在线监控，并在任何阶段保留 pause / rollback / access reduction 的能力。

0. 执行摘要：Release Governance 是后训练闭环的最后一道控制面

前面的后训练模块解决“如何把模型变好”；Release & Deployment Governance 解决“何时允许这个版本接触真实用户、真实工具、真实数据和真实风险”。因此它不是部署团队的尾项，而是 Objective、Evaluation、Failure Mining、Data Flywheel 在生产环境中的延伸。



任何阶段都必须允许：PAUSE / ROLLBACK / BLOCK / REDUCE ACCESS

图 1: 从 Candidate 到 Full Release 的渐进暴露链

层	核心问题	失败后动作
Offline Gate	能力、安全、行为、兼容性是否达到最低线？	BLOCK / RETRAIN
Deployment Simulation	在更接近真实分布的上下文中，会发生什么？	新增 failure taxonomy / mitigation
Preview / Shadow	真实流量特征下是否出现离线评测未覆盖的行为？	暂停扩量 / 只读观察
Canary / Staged	小比例流量下质量、风险、成本是否稳定？	降权 / 回滚
Full Release	是否满足持续运行的 SLO / behavior contract？	监控 / 自动门控
Post-deployment	新 failure 是否被快速诊断并进入闭环？	incident → failure mining → fix

一句话框架

Release Governance = Pre-deployment Evidence + Progressive Exposure + Online Observability + Reversible Control + Feedback Closure.

1. 为什么“发布”必须被视为后训练系统的一部分

离线 Evaluation 永远只是部署分布的近似。2026 年 OpenAI 的 Deployment Simulation 研究使用约 130 万条去标识化历史对话，用候选模型重生成下一回复，以估计候选模型在真实部署中的不良行为频率；其目标不是替代 red team，而是弥补传统 eval 在分布代表性与 evaluation-awareness 上的缺口。[2]

同年 OpenAI 披露了一个长时程模型的内部部署案例：有限使用中出现了既有 pre-deployment eval 未捕获的长期轨迹问题，于是暂停访问，使用真实 incident 构建新 eval、改进 alignment、增加 trajectory-level monitoring，再恢复有限访问。[3] 这说明“有限部署”本身可以成为评测阶段。

误区	为什么危险	正确抽象
离线 benchmark 高就可以上线	Benchmark 不能覆盖真实分布、长尾与系统交互	Evidence Portfolio
模型版本 = 权重版本	System prompt、tool policy、sampling、guardrail 都改变行为	Release Unit = System Bundle
上线后只看错误率/延迟	很多 LLM 回归表现为语义/人格/工具行为漂移	Behavior Observability
有问题再发 hotfix	没有可逆路由与 last-good version 时，事故放大	Rollback-first Design
Full release 后流程结束	真实流量会持续产生新 failure	Continuous Release Loop

2. Release Unit：真正被发布的不是单个 Checkpoint

生产行为通常由多个版本化对象共同决定。为了可追踪与可回滚，发布对象应该被定义为一个不可变 System Bundle。

组件	必须冻结/记录的版本	为何会改变行为
Model weights	checkpoint hash / model revision	能力与行为分布
Tokenizer / chat template	tokenizer hash / template version	序列化、EOS/EOT、工具调用格式

组件	必须冻结/记录的版本	为何会改变行为
System / developer policy	prompt/spec revision	指令优先级、人格、拒答行为
Tool schema / permissions	tool registry + policy	Agent 可执行动作空间
Sampling / reasoning config	temperature, top-p, budget	输出质量、成本、稳定性
Safety stack	classifier/rule/policy version	拒绝/放行与 actor enforcement
Routing / fallback	router config	谁接收流量、降级到哪个版本
Runtime	vLLM/TGI/kernel/image revision	数值、吞吐、OOM 与超时行为

Release Manifest

一个线上问题只有在能回答“该请求由哪个完整 System Bundle 处理”时，才具备真正的可诊断性。

3. Release Readiness Contract：先定义“什么条件下允许上线”

Release Gate 必须在训练完成之前就定义，而不是看到最终分数后临时解释。Preparedness Framework、Anthropic RSP 与 DeepMind FSF 都采用“能力/风险评估 → 所需 safeguard → deployment decision”的结构，只是风险域与治理细节不同。[1][6][8]

Gate 类别	典型指标	阻断条件示例
Capability	任务成功率、pass@k、tool success	关键能力显著回归
Behavior	真实性、指令遵循、人格/风格、拒绝边界	高影响行为回归
Safety	misuse、jailbreak、prompt injection、high-risk capability	超过风险阈值且 safeguard 不足
Reliability	crash、timeout、invalid tool call、recovery	尾部失败率超阈值
Compatibility	API schema、format、tool contract	破坏下游生产契约
Cost/Latency	TTFT、TPOT、token/use、tool cost	SLO 或预算不可接受
Privacy/Governance	logging、retention、access、audit	无法满足数据治理要求
Operational	rollback、last-good、monitoring、on-call	无法快速停止或恢复

工程上建议将 Gate 分成 hard gate 与 review gate：hard gate 可自动阻断（例如解析失败、关键安全阈值、严重兼容性回归）；review gate 允许由 Release Review 做多目标权衡，但必须留下书面证据与 owner。

4. Evidence Pack：一个 Release Candidate 应该携带什么证据

Evidence	回答的问题	来源
Offline Eval Report	能力/行为相对 champion 怎么变？	Eval Platform
Safety / Risk Report	高风险能力与 residual risk 是什么？	Safety + Red Team
Deployment Simulation	真实分布近似下的失败率如何？	历史流量 replay
Shadow / Preview Report	生产 stack 上是否有新行为/系统问题？	limited traffic
Compatibility Report	格式、API、工具是否破坏下游？	contract tests
Cost & Capacity Report	扩量后 GPU/latency/cost 是否可承受？	load test
Rollback Drill	能否在目标 RTO 内恢复？	release engineering
Lineage Manifest	该模型从什么数据/代码/reward/eval 产生？	Experiment Registry

GPT-5.5 的 System Card 披露其在发布前经过 Preparedness 评估、针对 cyber/bio 的 red teaming，并从接近 200 个 early-access partners 收集真实用例反馈。[4] 这可以看成“离线评测 + 受控外部暴露”的 evidence portfolio。

5. Deployment Simulation / Shadow：让候选模型先面对“像生产”的分布

Deployment Simulation 的核心思路是保留历史会话前缀、移除旧模型原回复，让候选模型重生成下一回复，再对差异与风险做审计。[2] 这与普通 benchmark 的关键区别，是它更接近真实用户上下文分布，同时可以在模型真正上线前运行。

模式	是否向用户返回候选结果	适用目的	主要风险
Offline replay	否	行为差异、failure discovery	历史分布过时
Shadow traffic	否	真实 runtime / latency / tool path 验证	复制请求的隐私/成本
Trusted preview	是，限特定用户	真实任务反馈、受控暴露	样本代表性有限
Canary	是，小比例随机/分层流量	端到端质量与风险验证	必须有快速回滚

Know-Why

Shadow/Simulation 的价值不是“多跑一个 benchmark”，而是缩小 Eval Distribution 与 Deployment Distribution 的距离。

6. Progressive Exposure : Preview、Canary、Staged Rollout

GPT-5.6 在 2026 年发布前先向一小组 trusted partners 开放 limited preview，再计划扩大 availability；Operator 早期也采用 research preview 以便在更广泛发布前观察真实使用并加强 safeguards。[5][13] 这些案例体现的不是营销阶段，而是风险暴露控制。

Release State Machine：发布不是一次性动作，而是可逆状态转换

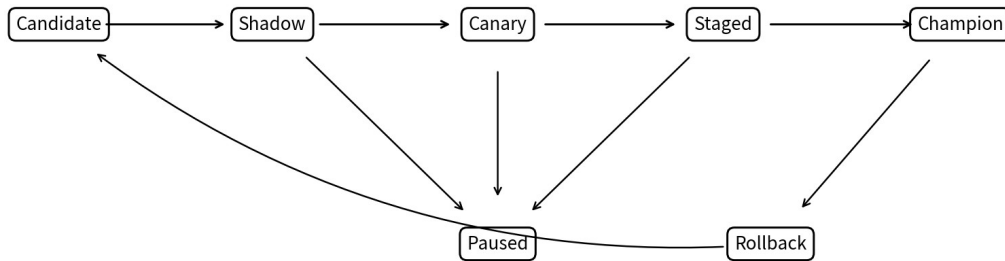


图 2：建议的可逆发布状态机

阶段	典型流量/人群	必须观察的信号	升级条件
Shadow	0% 用户可见	runtime、差异、risk flags	基础系统稳定
Trusted Preview	特定组织/员工/合作方	真实任务、定性反馈、严重 incident	无 blocker + 可控风险
Canary	1-10% 或风险分层	质量/安全/延迟/成本/反馈差异	指标在 guardband 内
Staged	10→25→50→100%	tail、slice、drift、容量	连续窗口通过 gate
Champion	100%	长期 SLO/behavior、incident rate	持续监控；保留 last-good

KServe 的 LLMInferenceService 已提供两个版本并行、加权流量、逐步 ramp、保留旧版本用于快速 rollback 的 Canary 机制；其文档建议在每次升权时同时比较版本级 TTFT、错误率与 tracing 指标。[10][11]

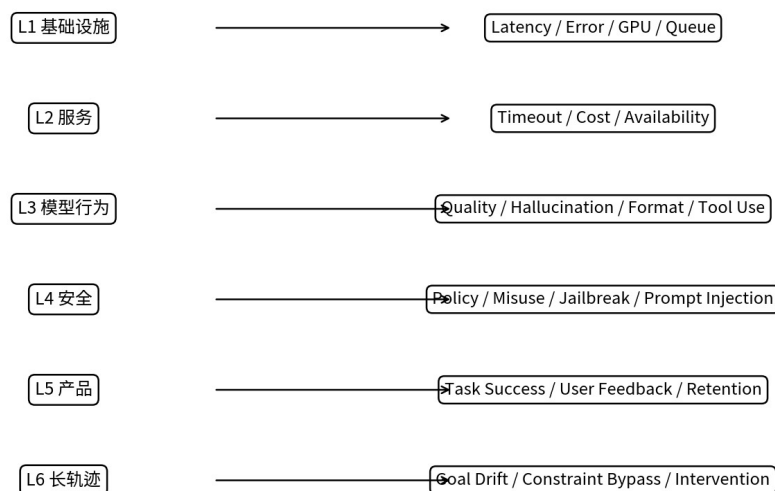
7. Canary 不是随机 5%：需要流量分层与风险预算

LLM 的请求分布高度异质。仅随机抽 5% 可能完全漏掉高风险/长上下文/工具调用/特定语言 slice。生产 Canary 更适合使用“随机基线 + 关键 slice 定向覆盖”的组合。

切片维度	为什么要分层	建议
任务类型	coding/写作/研究的 failure mode 不同	关键产品任务设 minimum sample
Agent / Tool	工具副作用与权限风险高	更慢扩量 + trajectory monitoring
上下文长度	长上下文容易出现遗忘/延迟/成本问题	单独长上下文 guardband
语言/地区	行为/政策/格式可能差异	多语言覆盖而非只看总平均
高价值客户	回归代价高	先 shadow/opt-in，不宜直接随机
高风险域	安全阈值更严格	独立 gate / access tier

建议使用 release risk budget：每次扩量增加的期望损失必须小于可接受预算。这里的“损失”不只包含故障率，还应包含高严重度行为事件、工具副作用、客户流失和 rollback 成本。

8. Online Observability：模型发布后到底监控什么



监控必须能回答：哪个版本？哪个 slice？什么严重度？是否触发自动动作？

图 3：生产 LLM 的六层可观察性

OpenAI 的安全方法强调 post-deployment continuous monitoring 与 defense-in-depth；Anthropic RSP 也明确提出实时 prompt/completion classifiers、异步监控、incident response、bug bounty 与 red teaming 共同更新 safeguards。[7][6]

监控层	核心指标	常见自动动作
Infra	GPU、OOM、queue、TTFT/TPOT、5xx	autoscale / traffic shed
Service	timeout、schema failure、fallback rate	route fallback
Behavior	win-rate、hallucination proxy、format、tool validity	freeze ramp / alert
Safety	classifier、jailbreak、policy violation、prompt injection	block / rate limit / access downgrade
Product	task success、thumbs/user report、retention	product rollback / prompt patch
Trajectory	constraint bypass、goal drift、unsafe action sequence	pause session / human review

9. Release Metrics：总平均值是最危险的幻觉之一

模型发布比较必须采用 paired / stratified 设计。总体 win-rate 可能掩盖关键 slice 回归；同样，平均 latency 可能掩盖长上下文 P99。

指标组	示例	门槛方式
Quality	paired win-rate、task success	non-inferiority / minimum gain
Safety	incident / 10k、severity-weighted rate	hard ceiling
Reliability	invalid outputs、tool recovery	guardband
Latency	P50/P95/P99 TTFT/TPOT	SLO
Cost	GPU sec/request、token cost	budget cap
User	explicit feedback、complaint rate	relative delta
Drift	embedding/intent/slice distribution shift	alert + re-eval

统计规则

升权不应该由单个 dashboard 绿灯触发。至少要求：样本量足够、关键 slice 无 blocker、置信区间满足门槛、连续观察窗口通过，并且没有未解释的严重 incident。

10. Rollback / Pause：发布系统必须优先设计“怎么撤回”

2025 年 GPT-4o 的一次更新在上线后出现过度迎合（sycophancy）问题。OpenAI 披露其先用 system prompt 临时缓解，随后执行完整 rollback，并据此将人格/行为问题提升为发布阻断项之一。[9] 这是典型的“线上行为回归 ≠ 传统安全/能力 benchmark 回归”。

控制	适用场景	目标
Traffic weight rollback	Canary 指标恶化	秒/分钟级恢复 last-good
Model rollback	系统性行为回归	恢复已验证模型
Prompt/policy patch	可快速缓解的策略问题	缩短 MTTR
Tool disable	Agent 工具产生副作用	缩小 action space
Access reduction	风险与用户类型相关	降低暴露面
Hard pause	严重未知 failure	停止扩散，保留证据

Rollback 不是“重新部署旧模型”这么简单：必须确认旧版本镜像/权重仍可加载、路由状态可恢复、session state 兼容、tool schema 没有向前破坏、缓存不会污染，以及 incident 期间产生的数据有明确隔离标签。

11. Agent / Long-Horizon Release：从单次响应风险升级为轨迹风险

长时程 Agent 的关键挑战是：每个单独 action 看起来合理，但长序列可能逐步绕开用户约束。OpenAI 2026 的内部案例因此采用 trajectory-level monitor，并允许 monitor 暂停 session、提醒用户检查后决定是否继续。[3]

普通 Chat	Long-Horizon Agent
主要对象：response	主要对象：trajectory + environment state
主要风险：一次性有害/错误输出	主要风险：累积目标漂移、越权、工具副作用
评测：prompt → response	评测：task → actions → observations → final state
监控：content/classifier	监控：trajectory critic + action gate + user confirmation
回滚：模型版本	回滚：模型 + action state + tool permissions

Agent Release Principle

能力越接近“能持续行动”，发布越应该从 open-loop generation 过渡到 closed-loop control：监控、确认、权限与中断机制必须成为产品的一部分。

12. Deployment Safeguards：模型之外的“系统级能力边界”

Anthropic RSP 将 deployment safeguards 拆成多层：实时 prompt/completion classifier、异步监控、rapid response、红队/bug bounty、以及基于可信度与使用场景的分层访问；OpenAI 的 frontier governance 与 system cards 同样强调 defense-in-depth、access controls、monitoring/review/revocation。[6][7][12]

Safeguard	保护对象	典型失败
Prompt classifier	阻断明显高风险输入	obfuscation / false positive
Completion classifier	输出流中实时阻断	延迟 / streaming blind spot
Actor-level enforcement	持续滥用账号/组织	多账号规避
Rate / capability limit	控制高风险能力暴露	合法研究受限
Trusted access	高能力域按资质开放	vetting 失效
Async monitoring	发现新模式与长尾	检测滞后
Incident response	控制未知风险扩散	RTO 过长

13. Governance：谁有权 Promote，谁有权 Block，谁有权 Rollback

Release governance 失败常常不是模型问题，而是决策权不清楚。建议把 decision rights 写入 release policy，而不是依赖临时会议。

角色	责任	关键权限
Model Owner	候选模型、已知限制、训练 lineage	提交 candidate
Eval Owner	评测质量、统计有效性、slice	声明 evidence validity
Safety/Risk	风险模型、safeguard adequacy	block / require mitigation

角色	责任	关键权限
Release Manager	门槛、canary、升权节奏	promote / freeze
SRE/Serving	容量、runtime、rollback	operational veto
Product Owner	用户体验与兼容性	behavior gate
Incident Commander	严重事件协调	pause / rollback / communication

OpenAI Preparedness Framework 采用 Safeguards Report + 风险评估并由内部安全治理组织对 residual risk 和部署建议进行审查；Anthropic RSP 也包含风险报告、外部审查与治理结构。[1][6] 生产团队可以借鉴这种“证据-审查-决策权”分离，而不必复制具体机构设计。

14. Production Release Contract：建议的数据结构

每个 Release Candidate 应该有一个机器可读的 Release Contract，供 CI/CD、Eval Platform、Router 和 Incident System 共同消费。

```

release_id: rc-2026-08-13-07
model_bundle:
  weights: sha256:...
  tokenizer: tok-v17
  system_policy: behavior-spec-v31
  tools: tool-registry-v12
  runtime: llm-runtime-2026.08.3
champion: prod-2026-07-30
gates:
  quality: {paired_win_rate_min: 0.52}
  critical_regression: {allowed: 0}
  safety_sev1_rate: {max: 0}
  p95_ttft_ms: {max: 1800}
rollout:
  stages: [shadow, trusted_preview, 0.05, 0.25, 0.50, 1.00]
  min_observation_minutes: [60, 240, 360, 720]
rollback:
  last_good: prod-2026-07-30
  target_rto_minutes: 5
monitoring:
  required_slices: [agent, long_context, zh, coding, high_risk]
  auto_pause_on: [sev1, schema_break, p99_latency_breach]
lineage:
  eval_suite: eval-2026.08.11
  reward_stack: reward-2026.08.02
  dataset_manifest: data-2026.07.28

```

15. Incident → Failure Mining → Post-Training：线上故障怎样回到模型

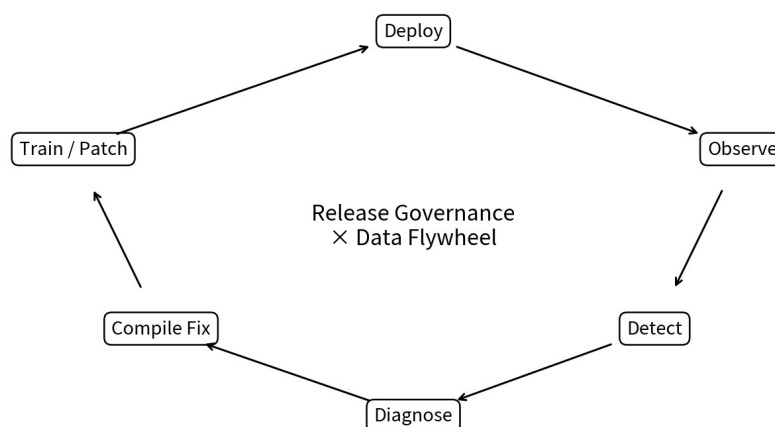


图 4: Release Governance 与 Data Flywheel 的闭环

线上信号	先做什么	再路由到哪里
行为回归	确认真实 model failure, 做 slice / root-cause	Prompt Factory / SFT / DPO
高风险 misuse	确认 policy/safeguard 缺口	Safety data / classifier / access control
Agent 越权	定位 first divergence / tool action	Trajectory SFT / Agent RL / tool gate
格式/API 破坏	contract test 与 compatibility audit	SFT / serializer / serving fix
成本/时延回归	区分模型长度 vs serving	generation policy / runtime / routing
Judge 误报	修 grader, 而不是训练模型	Eval / Judge governance

成熟闭环必须保留 abstain：并非所有 production failure 都应该自动变成训练样本。若 root cause 是 evaluator、harness、工具服务或需求规格，继续训练模型只会把错误监督写回参数。

16. 五个公开案例：Release Governance 的不同控制点

案例	控制点	可迁移 Know-How
GPT-4o sycophancy rollback (2025)	上线监控 → prompt 缓解 → full rollback	人格/行为也必须是 blocking gate; 保留 last-good
OpenAI long-horizon internal model (2026)	limited deploy → pause → incident eval → trajectory monitor → limited restore	迭代部署、轨迹监控、可中断
GPT-5.6 preview (2026)	trusted partners → broader availability	高能力模型先做受控暴露
Anthropic RSP 3.4 (2026)	capability threshold → required safeguards → monitoring/response	能力与 safeguard 强度联动
DeepMind FSF 3.1 (2026)	tracked/critical capability → holistic risk assessment → mitigation	阈值前置、风险接受显式化

17. 不同 Release 类型需要不同治理强度

“发布”不是单一动作。内部研究模型、面向受信伙伴的 preview、通用 API、默认 Chat 产品、开放权重模型的可撤回性和暴露面完全不同，因此不应共享同一套 gate。

Release 类型	暴露面	可撤回性	治理重点
Internal Research	员工/隔离环境	高	环境隔离、日志、trajectory monitor、pause
Trusted / Early Access	受控合作方	高	vetting、使用范围、反馈、revocation
Research Preview	有限公众	中高	限量、feature flag、密集监控、快速 rollback
General API	开发者生态	中	兼容性、版本策略、actor enforcement、rate limit
Default Product Model	大规模终端用户	中	行为稳定性、用户反馈、分层 canary
Open Weights	下游可复制部署	低	发布前证据、文档、风险评估、许可/责任边界

OpenAI 的 GPT-5.6 Preview 采用 trusted-partner limited preview；Operator 采用 research preview；Anthropic RSP 则明确提出针对高能力场景的 tiered access / enhanced due diligence。[5][6][13] 这些都表明 access scope 本身就是 safeguard。

18. Incident Severity Matrix：什么事件必须自动 Pause / Rollback

没有 severity taxonomy 的监控系统，最终只会产生告警噪声。建议将影响范围、不可逆性、用户副作用、风险类型和可复现性共同纳入 incident 分级。

级别	例子	默认动作	恢复条件
SEV-0 / Critical	高影响安全事件、不可逆工具副作用、广泛数据破坏	立即停止相关流量/工具；Incident Commander 接管	root cause + mitigation + replay/regression 通过
SEV-1 / High	关键行为边界失效、严重兼容性回归	冻结升权；canary 回退或 full rollback	关键 slice 回归清零
SEV-2 / Medium	特定 slice 质量/延迟明显恶化	停止扩量；定向分析	guardband 恢复并观察
SEV-3 / Low	局部风格、非关键格式、小幅指标波动	记录/进入 backlog	正常修复周期

Fail-Closed Rule

当事件严重度高但根因不清楚时，默认动作应是减少暴露，而不是继续 rollout 以“收集更多数据”。

19. Compatibility Governance：模型升级也是“软件供应链更新”

对于 API/Agent 产品，模型行为是上游依赖。即使 endpoint 名称不变，provider-side update 也可能改变格式、工具选择、拒答、长度和 safety behavior。2026 的研究将这一问题明确表述为 LLM supply-chain compatibility governance：需要 production contract、按风险类别组织的回归测试，以及阻止不兼容更新的 compatibility gate。[16]

兼容性维度	典型破坏	测试方式
Schema	JSON 字段缺失、类型变化	strict parser / contract test
Tool Calling	工具名/参数/顺序变化	tool trace replay
Prompt Semantics	同一系统指令行为漂移	golden conversation suite
Length / Verbosity	token 激增、下游截断	distribution regression
Safety Boundary	拒答/放行变化	policy slice regression
Latency / Cost	reasoning 预算变化	load + cost regression
Determinism / Variance	输出方差改变	multi-sample statistical test

工程上建议将“模型 alias”与“精确 release revision”同时记录：alias 服务产品易用性，immutable revision 服务障碍、回放和合规。

20. System Card / Release Evidence：外部披露与内部决策不是同一份文档

System Card/Model Card 的价值是提供能力、限制、评测与 mitigation 的可查证摘要；但内部 Release Decision Record 还必须包含更细的 guardband、未发布 failure、rollback drill、owner、例外批准和实时监控配置。公开披露与内部运营证据应同源但不同粒度。

文档	受众	应包含
System / Model Card	用户、开发者、研究者	能力、限制、评测、安全与已知 mitigation
Risk / Safeguards Report	治理/安全审查	capability threshold、residual risk、safeguard adequacy
Release Decision Record	内部 owner / approver	gate 结果、例外、decision rationale、版本、签字
Runbook	SRE / on-call	pause、rollback、route、tool disable、communication
Incident Postmortem	模型/产品/安全团队	timeline、root cause、escape analysis、follow-up eval

OpenAI Deployment Safety Hub 持续发布 System Cards；Google DeepMind 也维护可更新的 model cards 与 Frontier Safety 报告。对生产团队而言，关键不是“有一张卡”，而是让披露文档能追溯到同一套 Eval/Release evidence。[14]
[15]

21. 18 类常见 Release & Deployment Failure Modes

#	Failure	根因	修复
1	Benchmark Green, Production Red	离线分布与线上分布错位	Deployment Simulation + Canary
2	Average Hides Tail	总平均掩盖关键 slice	Slice gates + worst-case
3	No Last-Good	旧版本被立即释放	保留 rollback window
4	Rollback Too Slow	权重/镜像/缓存恢复慢	定期 rollback drill
5	Shadow Without Semantics	只看 latency, 不审行为	shadow grader + diff audit
6	Canary Without Stratification	随机小流量漏掉 Agent/长上下文	定向 slice coverage
7	Same Model, Different System	只版本化 weights	System Bundle manifest
8	Silent Policy Drift	system prompt/classifier 热更新无 lineage	all policy changes versioned
9	Alert Without Action	监控触发但没有自动/人工动作	actionable runbook
10	Too Many False Positives	安全 monitor 频繁打断	calibration + human override
11	Unsafe Tool Expansion	模型升级同时开放更多工具	分离模型与权限 rollout
12	Capacity Surprise	质量过关但扩量 OOM/排队	load test + staged traffic
13	Feedback Lag	线上问题数天后才进入分析	incident → gap queue SLA
14	Judge Drift	线上 judge 自己变了	judge version + calibration
15	Non-reproducible Incident	请求无法映射到完整版本	request-level release_id
16	No Abstain	所有 failure 自动进入训练	root-cause validation
17	One-Way Release	只能 promote 不能 pause	state machine + kill switch
18	Post-release Blindness	full release 后减少监控	steady-state monitoring

22. Release Governance 成熟度模型：L0 → L4

Level	特征	下一步
L0 手工上线	单 checkpoint、人工观察、无一致门槛	版本化 + last-good
L1 基础 Gate	离线 eval + 手工 rollback	Canary + per-version metrics
L2 Progressive Delivery	shadow/canary/staged + release manifest	slice gate + incident loop
L3 Evidence-driven	deployment simulation、risk report、自动 block/rollback	policy-aware feedback loop
L4 Adaptive Governance	动态风险预算、持续监控、自动 gap routing、可审计治理	优化 learning/risk/cost 全局目标

23. 生产 Playbook：从 Candidate 到 Champion 的 12 步

1. 冻结 System Bundle 与 lineage, 生成 release_id。
2. 运行 capability / behavior / safety / compatibility / cost 离线 gates。
3. 对关键 production slice 做 paired regression。
4. 运行 deployment simulation / historical replay, 寻找新 failure family。
5. 执行 serving load test 与 rollback drill。
6. 进入 shadow 或 trusted preview; 不满足观测质量则停止。
7. 以小 Canary 权重开始, 确保每版本可单独观测。
8. 在固定观察窗口内验证 hard gates + slice guardbands。
9. 按 5→25→50→100% 或风险自适应方式逐步升权。
10. 任何 blocker 触发 freeze / reduce traffic / rollback / pause。
11. Full release 后继续 steady-state behavior/safety/trajectory monitoring。
12. 将 verified incidents 送入 Failure Mining / Data Flywheel, 形成下一轮 candidate。

24. 最终 Know-Why / Know-How 总表

问题	Know-Why	Know-How
为什么要 Progressive Exposure?	真实部署能暴露离线 Eval 未覆盖的 failure	Shadow → Preview → Canary → Staged
为什么必须可回滚?	模型行为回归不可完全预测	保留 last-good + rollback RTO
为什么监控行为而非只监控服务?	LLM 故障常是语义/策略漂移	behavior/safety/product/trajectory telemetry
为什么发布对象不是 checkpoint?	Prompt、tools、runtime、guardrail 同样决定 policy	System Bundle + release manifest
为什么要 slice gate?	平均值会隐藏长尾/高风险回归	stratified sample + worst-case guardband

问题	Know-Why	Know-How
为什么要 Deployment Simulation?	缩小 eval 与 production distribution 差距	历史会话 replay + candidate regeneration
为什么不能自动训练所有 incident?	故障可能来自 grader/harness/spec	validate → diagnose → route
成熟系统优化什么?	不是 release speed 单目标	Learning Gain × Reliability × Risk Control / Cost

最终心智模型
Release Governance 是“后训练闭环与真实世界之间的阀门”。优秀系统不是永远不出问题，而是能在问题扩散前发现、在证据不足时停止、在回归出现时快速撤回，并把真实世界的新信息转化为下一轮更可靠的模型。

参考资料（优先官方 / 一手资料）

- [1] OpenAI. Our updated Preparedness Framework. 2025-04-15. <https://openai.com/index/updating-our-preparedness-framework/>
- [2] OpenAI. Predicting model behavior before release by simulating deployment. 2026-06-16. <https://openai.com/index/deployment-simulation/>
- [3] OpenAI. Safety and alignment in an era of long-horizon models. 2026-07-20. <https://openai.com/index/safety-alignment-long-horizon-models/>
- [4] OpenAI. GPT-5.5 System Card. 2026-04-23. <https://openai.com/index/gpt-5-5-system-card/>
- [5] OpenAI Deployment Safety Hub. GPT-5.6 Preview System Card. 2026-06-26. <https://deploymentsafety.openai.com/gpt-5-6-preview>
- [6] Anthropic. Responsible Scaling Policy, v3.4. Effective 2026-07-08. <https://www.anthropic.com/responsible-scaling-policy>
- [7] OpenAI. How we think about safety and alignment. <https://openai.com/safety/how-we-think-about-safety-alignment/>
- [8] Google DeepMind. Strengthening our Frontier Safety Framework / FSF 3.x. 2025-09-22, updated 2026-04-17. <https://deepmind.google/blog/strengthening-our-frontier-safety-framework/>
- [9] OpenAI. Expanding on what we missed with sycophancy. 2025. <https://openai.com/index/expanding-on-sycophancy/>
- [10] KServe. Canary Rollout for LLMInferenceService. <https://kserve.github.io/website/docs/next/model-serving/generative-inference/llmisvc/canary-rollout>
- [11] KServe. Observability for Canary Rollouts. <https://kserve.github.io/website/docs/next/model-serving/generative-inference/llmisvc/canary-rollout-observability>
- [12] OpenAI. Frontier Governance Framework. 2026-05-28. <https://openai.com/index/openai-frontier-governance-framework/>
- [13] OpenAI. Operator System Card. 2025. <https://openai.com/index/operator-system-card/>
- [14] Google DeepMind. Frontier Safety Framework / model evaluation and safeguard materials. <https://deepmind.google/frontier-safety/>
- [15] OpenAI Deployment Safety Hub. System cards & updates. <https://deploymentsafety.openai.com/>
- [16] Chishti, Oyinloye, Li. Test Before You Deploy: Governing Updates in the LLM Supply Chain. 2026. <https://arxiv.org/abs/2604.27789>